

# TP N°1

## Vision artificielle – Traitement d'images

Prepared for :

- Mr SOUALAH

Prepared by :

- Anis BENINI

# Sommaire

|                                                  |          |
|--------------------------------------------------|----------|
| <b>I. Introduction.....</b>                      | <b>3</b> |
| <b>II. Analyse des parties.....</b>              | <b>3</b> |
| Partie 1 : Préparation.....                      | 3        |
| Partie 2 : Implémentation de la Convolution..... | 4        |
| Partie 3 : Filtrage Non-Linéaire.....            | 7        |
| Partie 4 : Analyse et Comparaison.....           | 8        |
| <b>III. Conclusion.....</b>                      | <b>9</b> |

## **I. Introduction**

Ce rapport documente les travaux réalisés dans le cadre du TP N°1 sur la vision artificielle et le traitement d'images. Le projet a été réalisé en utilisant les bibliothèques Python suivantes :

- **NumPy** pour les manipulations matricielles
- **Matplotlib** pour l'affichage des images
- **PiL** pour le chargement et la manipulation des images
- **OpenCV** éventuellement pour des traitements avancés

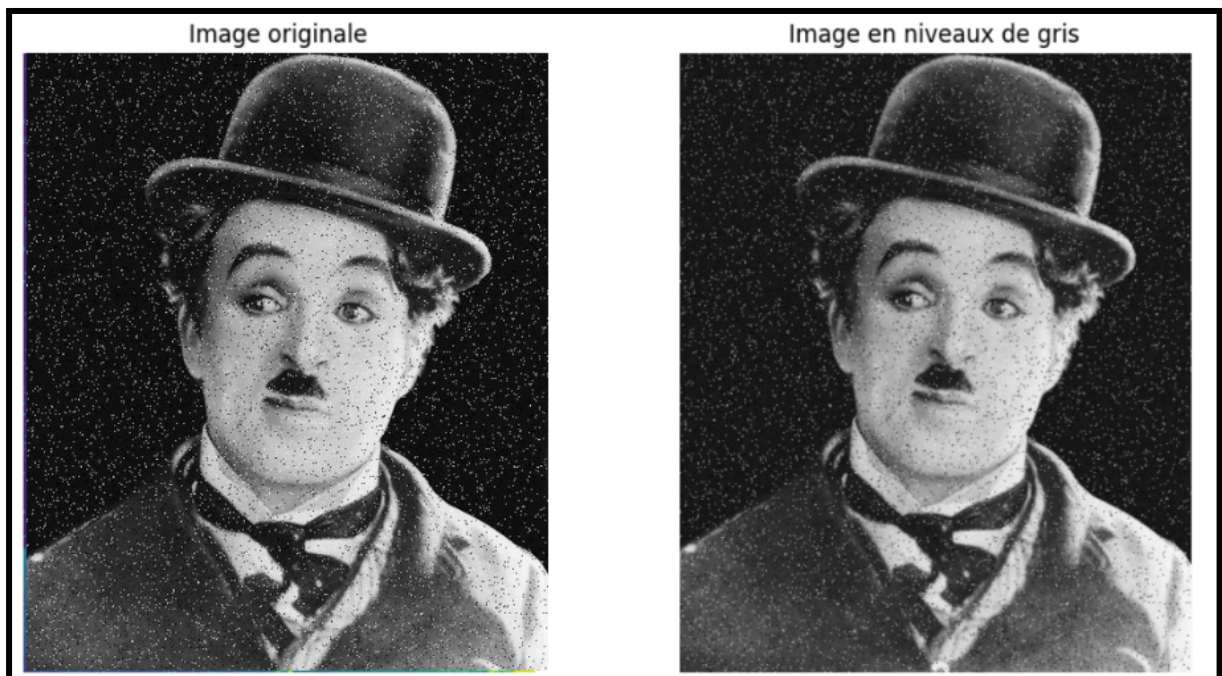
L'objectif principal était d'appliquer des techniques fondamentales de traitement d'images, y compris la convolution, le filtrage non linéaire et l'analyse des résultats.

---

## **II. Analyse des parties**

### **Partie 1** : Préparation

- Chargement de l'image :
  - Une image au format **.jpg** a été chargée à l'aide de la bibliothèque **PiL**.
  - L'image a été convertie en niveaux de gris pour simplifier les opérations de traitement.
  - Les images originales et en niveaux de gris ont été affichées à l'aide de **Matplotlib**.



- Représentation matricielle :
  - Les dimensions, les valeurs minimales et maximales des pixels sont bien calculées et affichées.
    - **Dimensions de l'image (619 x 512)** : Cela indique que l'image a une largeur de 619 pixels et une hauteur de 512 pixels, ce qui représente une image de grande taille.
    - **Valeurs des pixels : Min=0, Max=255** : Cela signifie que les valeurs des pixels dans l'image varient de 0 à 255, ce qui est généralement dans le cadre d'une image en niveaux de gris (valeurs entre 0 pour le noir et 255 pour le blanc, mais ici l'intensité des pixels atteint le maximum ).

Dimensions de l'image : 619x512  
Valeurs des pixels : Min=0, Max=255

## Partie 2 : Implémentation de la Convolution

- Implémentation de la fonction de convolution :
  - La convolution consiste à appliquer un noyau sur chaque pixel de l'image en calculant un produit matriciel entre le noyau et un patch local. La fonction de convolution a été implémentée en Python en ajoutant un **padding** à l'image via **np.pad**.

```

1  def convolution(image, kernel, padding_type="zero"):
2
3      kernel = np.flipud(np.fliplr(kernel)) # Retourne le noyau
4      kh, kw = kernel.shape
5      pad_h, pad_w = kh // 2, kw // 2
6
7      #ajouter du padding
8      if padding_type == "zero":
9          padded_image = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode="constant")
10     elif padding_type == "reflect":
11         padded_image = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode="reflect")
12     else:
13         raise ValueError("Type de padding non supporté")
14
15     #créer une matrice pour le résultat
16     output = np.zeros_like(image)
17
18     #appliquer la convolution
19     for i in range(image.shape[0]):
20         for j in range(image.shape[1]):
21             local_patch = padded_image[i:i+kh, j:j+kw]
22             output[i, j] = np.sum(local_patch * kernel)
23
24     return output

```

- Application des filtres :
  - **Flou (blur)** : utilisation d'un noyau de moyenne (3x3 et 5x5) pour lisser l'image.
  - **Détection de contours** : application d'un noyau Laplacien.
  - **Accentuation des détails (sharpening)** : création d'un noyau pour renforcer les contours.

```

1  #noyaux prédéfinis
2  blur_kernel_3x3 = np.ones((3, 3)) / 9  # Flou 3x3
3  blur_kernel_5x5 = np.ones((5, 5)) / 25  # Flou 5x5
4  laplacian_kernel = np.array([[0, -1, 0], [-1, 4, -1], [0, -1, 0]]) # Détection de contours
5  sharpen_kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]]) # Accentuation des détails
6
7  #appliquer chaque filtre
8  image_blur_3x3 = convolution(image_array, blur_kernel_3x3)
9  image_blur_5x5 = convolution(image_array, blur_kernel_5x5)
10 image_edges = convolution(image_array, laplacian_kernel)
11 image_sharpen = convolution(image_array, sharpen_kernel)

```

- Les résultats obtenus pour chaque filtre ont été comparés et affichés ( visualiser à l'aide de Matplotlib ) :

```

1  #affichage
2  plt.figure(figsize=(10, 10))
3
4  plt.subplot(2, 2, 1)
5  plt.title("Flou (3x3)")
6  plt.imshow(image_blur_3x3, cmap="gray")
7  plt.axis("off")
8
9  plt.subplot(2, 2, 2)
10 plt.title("Flou (5x5)")
11 plt.imshow(image_blur_5x5, cmap="gray")
12 plt.axis("off")
13
14 plt.subplot(2, 2, 3)
15 plt.title("Contours (Laplacien)")
16 plt.imshow(image_edges, cmap="gray")
17 plt.axis("off")
18
19 plt.subplot(2, 2, 4)
20 plt.title("Détails accentués")
21 plt.imshow(image_sharpen, cmap="gray")
22 plt.axis("off")
23
24 plt.tight_layout()
25 plt.show()

```

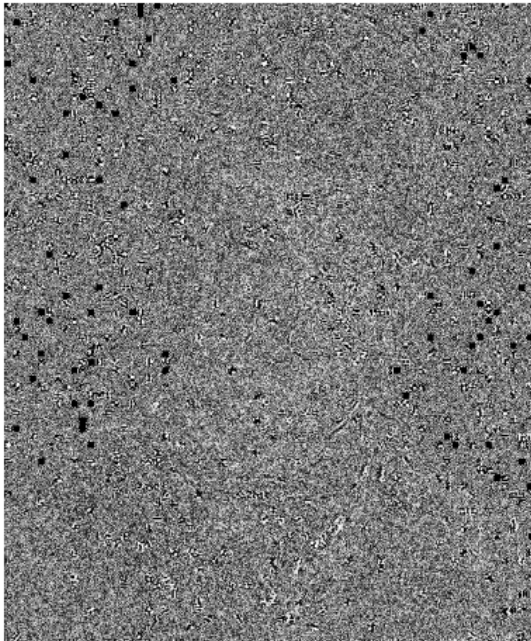
Flou (3x3)



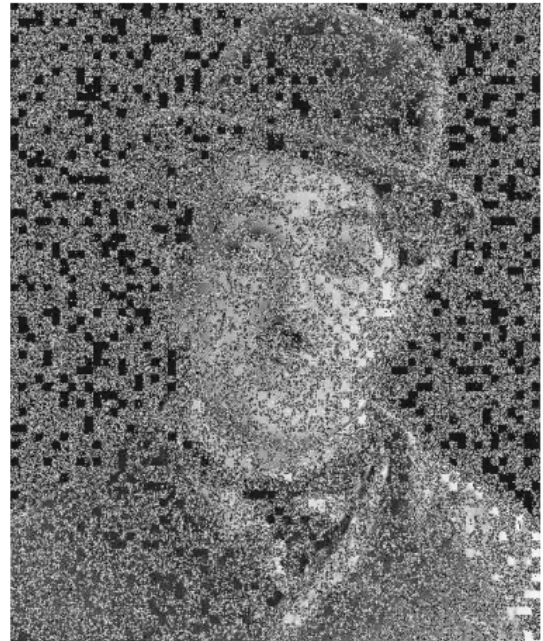
Flou (5x5)



Contours (Laplacien)



Détails accentués

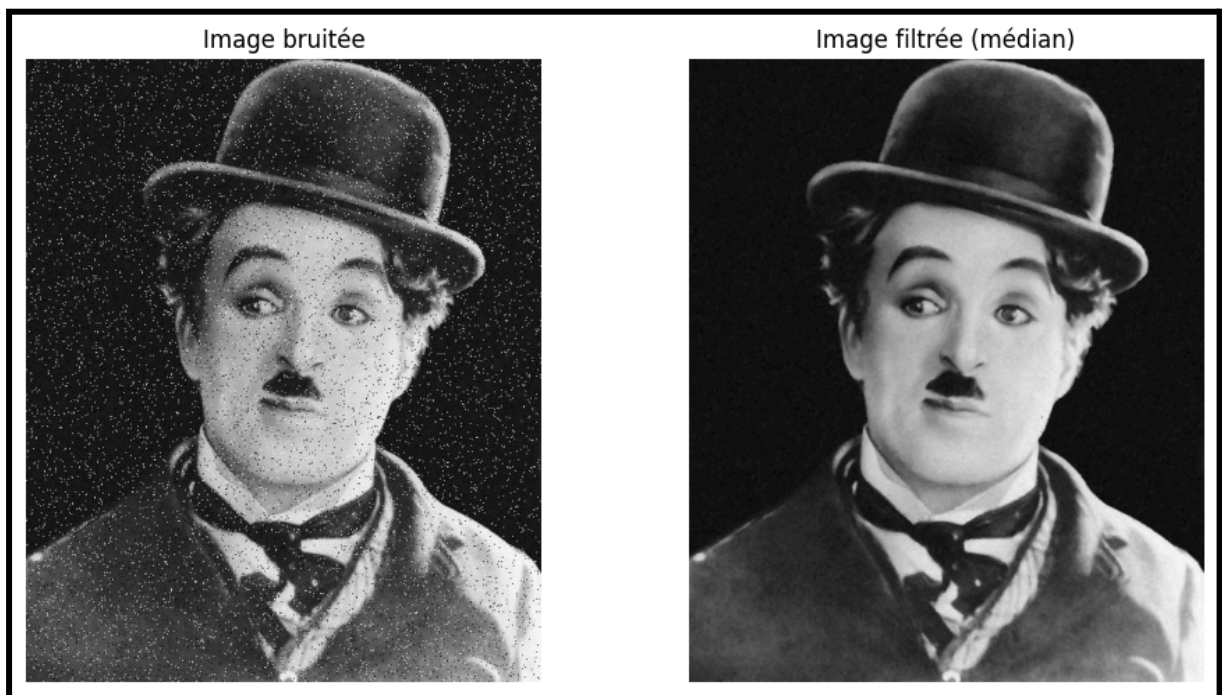


### Partie 3 : Filtrage Non-Linéaire

- Filtrage médian : Un filtre médian a été implémenté pour réduire le bruit impulsionnel (« sel et poivre »).  
Contrairement aux filtres linéaires, il préserve les contours tout en éliminant efficacement le bruit.  
Ce filtre remplace chaque pixel par la médiane des pixels dans un voisinage local.

```
1 from scipy.ndimage import median_filter
2
3 #appliquer le filtre médian
4 image_noisy = image_array.copy() # Ajouter du bruit si nécessaire
5 image_median = median_filter(image_noisy, size=3) # Taille du voisinage : 3x3
6
```

- Affichage :



## Partie 4 : Analyse et Comparaison

- Comparaison des résultats :
  - Créer un tableau 2x3 pour les images
  - Afficher les images originales et filtrées côte à côte:

```
1 fig, axs = plt.subplots(2, 3, figsize=(18, 12)) # Crée un tableau 2x3 pour les images
```

```
1 # Image originale
2 axs[0, 0].imshow(image_array, cmap="gray")
3 axs[0, 0].set_title("Image originale", fontsize=14)
4 axs[0, 0].axis("off")
5
6 # Flou (3x3)
7 axs[0, 1].imshow(image_blur_3x3, cmap="gray")
8 axs[0, 1].set_title("Flou (3x3)", fontsize=14)
9 axs[0, 1].axis("off")
10
11 # Flou (5x5)
12 axs[0, 2].imshow(image_blur_5x5, cmap="gray")
13 axs[0, 2].set_title("Flou (5x5)", fontsize=14)
14 axs[0, 2].axis("off")
15
16 # Détection des contours (Laplacien)
17 axs[1, 0].imshow(image_edges, cmap="gray")
18 axs[1, 0].set_title("Contours (Laplacien)", fontsize=14)
19 axs[1, 0].axis("off")
20
21 # Détails accentués (Sharpen)
22 axs[1, 1].imshow(image_sharpen, cmap="gray")
23 axs[1, 1].set_title("Détails accentués", fontsize=14)
24 axs[1, 1].axis("off")
25
26 # Filtrage médian
27 axs[1, 2].imshow(image_median, cmap="gray")
28 axs[1, 2].set_title("Filtrage médian", fontsize=14)
29 axs[1, 2].axis("off")
30
31 # Ajustement de l'espacement
32 plt.tight_layout()
33 plt.show()
```

- **Résultats :**

En constate que :

**1. Image originale:** elle contient du bruit sous forme de points blancs et noirs répartis de manière aléatoire. Ce bruit peut affecter la qualité de l'image et gêner les traitements ultérieurs.

**2. Flou (3x3):** L'application d'un filtre de flou avec un noyau 3×3 réduit partiellement le bruit. L'image semble plus lisse, mais certains détails fins sont également légèrement atténués. Ce filtre est efficace pour un léger adoucissement de l'image.

**3. Flou (5x5):** Le filtre de flou avec un noyau 5×5 donne un effet de lissage plus marqué par rapport au noyau 3×3. Bien que le bruit soit davantage réduit, l'image perd en netteté et certains contours deviennent flous.

**4. Contours (Laplacien):** L'application du filtre laplacien met en évidence les contours en détectant les variations d'intensité dans l'image. Cependant, en présence de bruit, de nombreux faux contours apparaissent, ce qui peut rendre l'interprétation difficile.

**5. Détails accentués:** Cette technique tente d'améliorer les détails de l'image en augmentant les contrastes. Toutefois, le bruit est également amplifié, ce qui peut nuire à la qualité globale de l'image.

**6. Filtrage médian:** Le filtrage médian est particulièrement efficace pour réduire le bruit impulsif (sel, poivre). Il conserve mieux les contours tout en éliminant une grande partie du bruit. L'image obtenue est plus propre et les détails sont mieux préservés par rapport aux filtres de flou.

- Mes réponses concernant les questions posé :

- Quels filtres sont les plus efficaces pour supprimer le bruit ?
    - Le filtre **médian** est le plus efficace contre le bruit impulsif, car il élimine le bruit tout en préservant les contours et les détails.
  - Quels filtres préservent mieux les contours ?
    - Les filtres comme le **Laplacien** ou **l'accentuation des détails** préservent et amplifient les contours, bien que le filtre **médian** les conserve également tout en supprimant le bruit.
  - Quels défis avez-vous rencontrés pour traiter les bords ?
    - **Effets de bord** : Lors de la convolution, la gestion des bords (padding) peut introduire des artefacts ou des distorsions, en particulier pour des filtres avec des noyaux de grande taille
    - **Amplification du bruit** : Les filtres comme le **Laplacien** ou **l'accentuation des détails** amplifient également les artefacts de bruit, ce qui peut être problématique.
  - Point à améliorer :
    - Tester des noyaux de taille différente pour adapter l'effet des filtres à l'image.
    - Appliquer un prétraitement pour réduire le bruit avant d'utiliser des filtres sensibles comme le Laplacien ou le Sharpen.
    - Expérimenter avec d'autres types de padding comme la réflexion ou la duplication pour réduire les effets de bord.
-

### **III. Conclusion**

Ce TP sur le traitement d'images a permis d'explorer divers filtres pour améliorer la qualité des images bruitées, notamment à travers la convolution avec noyaux de flou et la détection de contours, montrant des résultats variés. Le filtrage médian s'est révélé particulièrement efficace pour réduire le bruit tout en maintenant les contours, mais des défis demeurent en matière de traitement des bords et d'optimisation des calculs. En fin de compte, cette expérience a enrichi la compréhension des techniques classiques de traitement d'images et les compromis nécessaires entre lissage et préservation des détails.